MEDISYNC HEALTHCARE

Cryptographically Verified Clinical & Prescription Platform

Comprehensive Project Report • Architecture Breakdown • Interview Master Guide

Live Demo: https://tamilkakashi.github.io/medisync-healthcare/ | Repository: github.com/tamilkakashi/medisync-healthcare

1. Executive Overview & 60-Second Interview Pitch

When the interviewer says: "Tell me about your project", use this structured opening pitch:

60-SECOND ELEVATOR PITCH

"MediSync is an enterprise healthcare platform that solves prescription tampering and dispensary fraud using Public Key Infrastructure (PKI). Whenever a physician prescribes medication, the payload is hashed with SHA-256 and digitally signed with their RSA-2048 private key. Pharmacies mathematically verify the hash against the doctor's public key before dispensing medications. It features 5 role-based portals (Doctor, Pharmacy, Hospital, Patient, Admin), full dark/light clinical themes, and an offline-resilient client mock engine."

Key Statistics & Specifications:

- **Frontend:** React 19.2, Tailwind CSS v4, Lucide React, Framer Motion, jsPDF
- **Backend Architecture:** Node.js/Express REST API + Dual-Engine Client-Side Mock Database with Axios custom adapter
- **Security Standards:** RSA-2048 Asymmetric Key Pairs, SHA-256 Digest Hashing, Role-Based Access Control (RBAC)
- **Deployment & CI/CD:** GitHub Actions workflow deploying to GitHub Pages with SPA 404 routing
- **Demo Access:** 5 pre-configured 1-click role accounts (password: 123456 across all profiles)

2. System Architecture & The 5 Core Portals

A. Doctor Portal (testsurgeon@lumina.sys - Dr. Marcus Vance)

- **Prescription Issuance:** Structured medication builder (drug name, dosage, frequency, duration, expiry date).
- **Digital Signing Engine:** Computes canonical SHA-256 hash and generates RSA signature stored with the prescription.
- **Filter & Status Controls:** Interactive status filters (All, Pending, Dispensed) with real-time success rate telemetry.
- **Official Medical PDF:** Exports cryptographic prescription report with QR code and physician verification proof.

B. Pharmacy Portal (testpharmacy@lumina.sys - Apex Pharmaceuticals)

- **Incoming Dispatch Feed:** Live queue of pending prescriptions issued across affiliated hospitals.
- **Cryptographic Integrity Verification:** Validates digital signature against doctor's public key; instantly flags altered dosages.
- **Dispensing Lifecycle:** 1-click medication dispensing with timestamped audit trail.

C. Hospital Administration Portal (testhospital@lumina.sys - Metro Health Central)

- **Practitioner Management:** Register new doctors, configure room/floor numbers, and issue cryptographic licenses.
- **Patient Assignment:** Link patients directly to attending physicians with automated patient notifications.
- **Dispensary Governance:** Review and approve licensed community pharmacies.
- **Mobile Module Switcher:** Horizontal swipeable module pill bar for tablets and smartphones.

D. Patient Portal (testpatient@lumina.sys - Eleanor Rigby)

- **Prescription Wallet:** Real-time overview of active and past medications with dosage instructions.
- **Attending Physician Directory:** Direct office location, department, room, and floor details.
- **Notification Center:** Real-time assignment updates with centered mobile screen-fit overlay.

E. Administrator Portal (admin@medisync.com - MediSync Admin)

- **User Account Approval:** Review pending staff registrations and approve credentials.
- **Audit Trail Ledger:** Tamper-evident logs of every login, prescription created, verified, and dispensed.

3. Core Engineering Concepts & Technical Innovations

1. Public Key Infrastructure (PKI) & Digital Signatures

Prescription data (patientId, medicines, diagnosis, expiryDate) is serialized and hashed with SHA-256. The hash is encrypted with the doctor's RSA-2048 private key. The pharmacy decrypts the signature using the public key and compares it to the recomputed hash. If even a single character was altered, the check fails.

2. Dual-Engine Network Adapter (Axios Custom Adapter)

Deploying to static hosts like GitHub Pages causes /api/* endpoints to 404 because no Node server runs on the static CDN. MediSync resolves this using an Axios adapter in api.js: when running on GitHub Pages or if the backend is down, it seamlessly routes requests to clientMockDb.js, which manages localStorage state with full CRUD parity.

3. Mobile Viewport & Touch-First Ergonomics

Engineered for real clinical mobile devices with fixed bottom navigation (MobileBottomNav.jsx), spring-animated slide-up drawer (MobileNavDrawer.jsx), safe-area notch padding (env(safe-area-inset-bottom)), and mobile-clamped dialogs.

4. Clinical Themes & HIPAA Privacy Safeguards

Includes Daylight Light, Deep Midnight Slate Dark (#070b12), and System Sync modes with 5 accent palettes. Features a HIPAA-compliant Patient Privacy Screen that blurs patient names on shared terminals until explicitly hovered.

4. Real Engineering Challenges Faced & Step-by-Step Solutions

Challenge 1: GitHub Pages 404 on API Login

- **Problem:** Localhost worked, but live GitHub Pages returned HTTP 404 on /api/auth/login because static CDN lacks a Node.js server.
- **Solution:** Built clientMockDb.js with localStorage persistence. Wrote a custom Axios adapter in api.js that detects static hostnames (github.io) and intercepts API requests without network errors.

Challenge 2: Status & Date Overlap / Button Unresponsiveness

- **Problem:** On doctor dashboard, patient names and dates were truncated, and clicking status badges did not update the view.
- **Solution:** Added min-w-[700px] table constraint with overflow-x-auto, whitespace-nowrap date protection, and converted status badges into interactive filter toggle buttons.

Challenge 3: Hospital Navigation Hidden on Mobile & Tablets

- **Problem:** Desktop sidebar had hidden xl:flex, completely removing module navigation on screens under 1280px.
- **Solution:** Added a horizontal swipeable pill bar (xl:hidden) right below the dashboard header with active tab glow.

Challenge 4: Patient Notification Dropdown Cut Off on Mobile

- **Problem:** Tapping the notification bell on mobile pushed 80% of the card off the left edge of the phone screen.
- **Solution:** Identified that absolute right-0 aligned a 288px dropdown to a 56px button on the left edge. Fixed by switching to fixed inset-x-4 top-24 max-w-sm mx-auto with a tap-outside backdrop overlay.

5. Top 25 Interview Questions & Winning Answers

Q1: Can you explain the high-level architecture of MediSync?

Answer: MediSync is a single-page application built on React 19 and Tailwind CSS v4, utilizing a dual-mode data layer: a Node/Express REST backend for full-stack deployment and an offline-first client mock engine for static hosting. It enforces Role-Based Access Control (RBAC) across 5 clinical roles and uses asymmetric cryptography (RSA-2048 + SHA-256) for prescription authenticity.

Q2: What is the exact workflow of digital prescription signing?

Answer: When a doctor creates a prescription, the core parameters are stringified and hashed with SHA-256. The hash is encrypted with the physician’s private RSA key to produce a digital signature. At the pharmacy, the signature is decrypted using the doctor’s public key and compared to the newly computed hash. A match guarantees integrity and non-repudiation.

Q3: Why did you use HashRouter instead of BrowserRouter?

Answer: On static hosts like GitHub Pages or S3, direct navigation to subpaths like /dashboard results in a 404 because no server rewrite rule exists. HashRouter keeps navigation within hash fragments (/#/dashboard) handled entirely client-side, eliminating 404 errors on refresh without complex server routing.

Q4: How did you solve the problem of running a full-stack app on a static host like GitHub Pages?

Answer: I implemented a custom Axios adapter in api.js. It checks whether the application is running on a static host like github.io. If so, it intercepts requests and routes them to clientMockDb.js, which manages data in localStorage and reproduces the complete server API contract, including cryptography and stats.

Q5: What makes your project mobile-ready?

Answer: I implemented touch-first navigation including a fixed bottom navigation bar (MobileBottomNav.jsx) on screens < 768px, a slide-up gesture-friendly drawer (MobileNavDrawer.jsx), safe-area notch padding (env(safe-area-inset-bottom)), and responsive table swipe indicators.

Q6: Tell me about a specific CSS positioning bug you solved.

Answer: In the Patient Dashboard, the notification dropdown used absolute right-0 w-72. On mobile, the bell icon sat on the left of the screen, so aligning the right edge of a 288px box to a 56px button pushed 232px off-screen to the left. I replaced it on mobile with fixed inset-x-4 top-24 max-w-sm mx-auto and a backdrop overlay.

Q7: How does your theme engine work?

Answer: It uses React Context (ThemeContext) and localStorage to manage Light, Dark, and System modes. In System mode, it listens to window.matchMedia for OS dark mode changes. It injects classes on the html root along with custom data-accent, data-compact, and data-font-size attributes for instant variable re-computation.

Q8: How do you ensure HIPAA privacy on clinical terminals?

Answer: We built a Patient Privacy Screen feature. When enabled in System Settings, patient names are blurred using CSS backdrop filter on shared hospital terminals and only reveal when explicitly hovered by clinical staff.

Q9: What is the difference between asymmetric and symmetric encryption?

Answer: Symmetric encryption uses the same key for both encryption and decryption (e.g. AES). Asymmetric encryption uses a mathematically linked pair: a public key for encryption/verification and a private key for decryption/signing (e.g. RSA). We use asymmetric keys for non-repudiation.

Q10: How do you protect against SQL / NoSQL injection and credential exposure?

Answer: We sanitize all inputs, use strict Mongoose schema validation, never commit .env files or tokens to git, hash passwords using bcrypt with 10 salt rounds, and strip private keys from all user responses.

6. Step-by-Step 3-Minute Live Demo Script

Phase 1: Login & Role Selection (30 seconds)

Open https://tamilkakashi.github.io/medisync-healthcare/ and point out the 1-Click Demo Profiles. Click Doctor (Dr. Marcus Vance).

Phase 2: Doctor Prescription Creation (60 seconds)

Click "New Prescription", select Eleanor Rigby, add Lisinopril 10mg, and click "Issue Prescription & Sign". Explain SHA-256 hash generation and RSA private key signing. Click the prescription row to show the official medical layout with cryptographic proofs.

Phase 3: Pharmacy Verification & Dispense (45 seconds)

Log out, click Pharmacy (Apex Pharmaceuticals), open the pending prescription, highlight the "Digital Signature Verified" badge, and click "Dispense Medication".

Phase 4: Patient View & Mobile Controls (45 seconds)

Log in as Patient (Eleanor Rigby), show the status updated to Dispensed, tap the notification bell to show the centered notification modal, and toggle Dark Mode to demonstrate clinical contrast.